Raspberry Pi Pico & VS Code Quick-Start Guide

Complete reference for writing, testing, and flashing MicroPython code on Windows

1. Checking Connection

Ensure the Pico is plugged in via USB and recognized by your computer. Open a terminal in VS Code (`Ctrl + ``) and run:

```
python -m mpremote connect list
```

You should see an entry like `COM3 E6625888177E352D 2e8a:0005 MicroPython`. If nothing appears, check the USB cable (must support data, not just power).

2. Two Ways to Run Code in VS Code

Method A: Run in RAM (Testing)

When to use: While writing, editing, and debugging code.

Shortcut: `Ctrl + Shift + B`

- Sends the current script directly into Pico's volatile RAM.
- Does *not* overwrite flash memory (saves flash wear).
- Output and `print()` statements stream live to the VS Code terminal.
- Press `Ctrl + C` in the terminal to halt execution.

Method B: Flash to Pico (Permanent)

When to use: When code is ready to run standalone without a PC.

Command Palette: `Ctrl + Shift + P` → `Tasks: Run Task` → `Upload current file to Pico as main.py`

- Copies your active script onto the Pico flash drive as `:main.py` and triggers a reset.
- The Pico automatically executes `main.py` every time it powers up.
- Unplug from PC and power with any 5V USB charger or battery pack.

3. Useful Terminal Commands (Cheatsheet)

Task	Command
Run a specific script	`python -m mpremote connect auto run path\to\script.py`
Open interactive REPL	`python -m mpremote connect auto repl` (exit with `Ctrl + ]`)
List files stored on Pico	`python -m mpremote connect auto fs ls`
Delete auto-run script	`python -m mpremote connect auto fs rm :main.py`
Soft-reset Pico	`python -m mpremote connect auto soft-reset`

4. Essential MicroPython Starter Patterns

A. Controlling the Onboard Green LED

```
from machine import Pin
# "LED" works on Pico W and modern MicroPython firmware (GP25 on older Pico boards)
led = Pin("LED", Pin.OUT)
led.value(1)       # Turn ON
led.value(0)       # Turn OFF
led.toggle()       # Toggle state
```

B. Reading a Push Button (Active HIGH with Internal Pull-Down)

Wiring: One button leg to **3V3(OUT)** (Pin 36), the opposite leg to **GP20** (Pin 26). No external resistor needed.

```
from machine import Pin
button = Pin(20, Pin.IN, Pin.PULL_DOWN)

if button.value() == 1:
    print("Button pressed!")
```

C. Clean Startup / Reset Routine

MicroPython pins keep their state when a script ends. Always reset hardware at the top of your scripts:

```
from machine import Pin
from time import sleep_ms

def reset_hardware():
    # Turn off onboard LED
    try:
        Pin("LED", Pin.OUT).value(0)
    except Exception:
        pass
    # Reset any specific output pins (e.g. GP12 - GP18)
    for p in [18, 17, 16, 15, 14, 13, 12]:
        Pin(p, Pin.OUT).value(0)

reset_hardware()
sleep_ms(100) # Small settle delay
```

5. Common Troubleshooting & FAQs

Device busy / Access Denied error: If `mpremote` reports that the serial port is busy, an active terminal session or REPL is holding the connection. Click into the open terminal in VS Code and press **`Ctrl + C`**, or close the terminal panel.

How to stop an infinite loop on the Pico: In the VS Code terminal running the script, press **`Ctrl + C`** twice. This sends a keyboard interrupt to MicroPython and returns control to the prompt.

Why run `python -m mpremote` instead of just `mpremote`? On Windows, Python packages installed via Microsoft Store Python place executable scripts outside the default Windows PATH. Calling via `python -m mpremote` is 100% reliable across all terminal sessions without needing manual PATH edits.